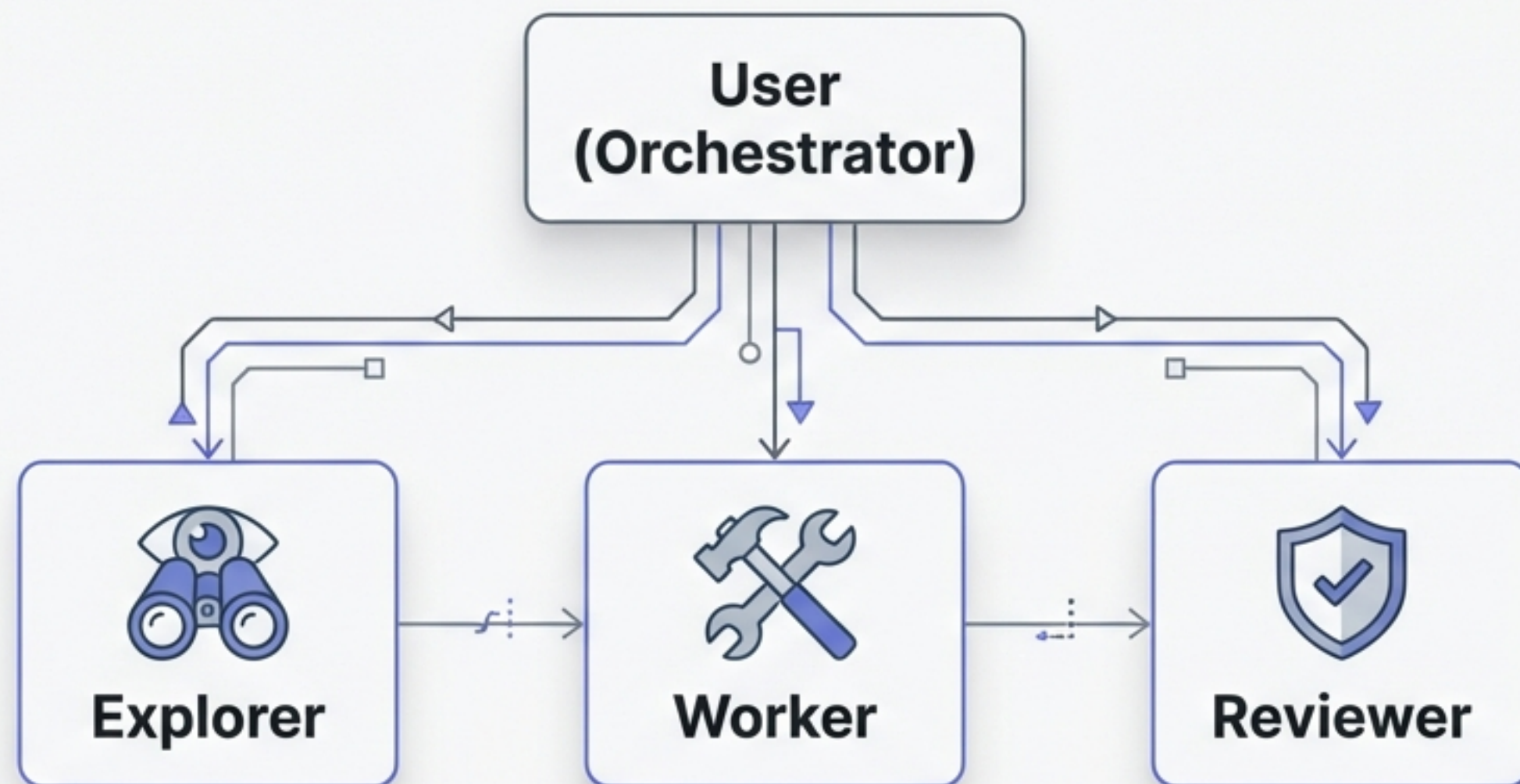


v0.102.0+ / Experimental Feature

Codex Multi-Agent Architecture

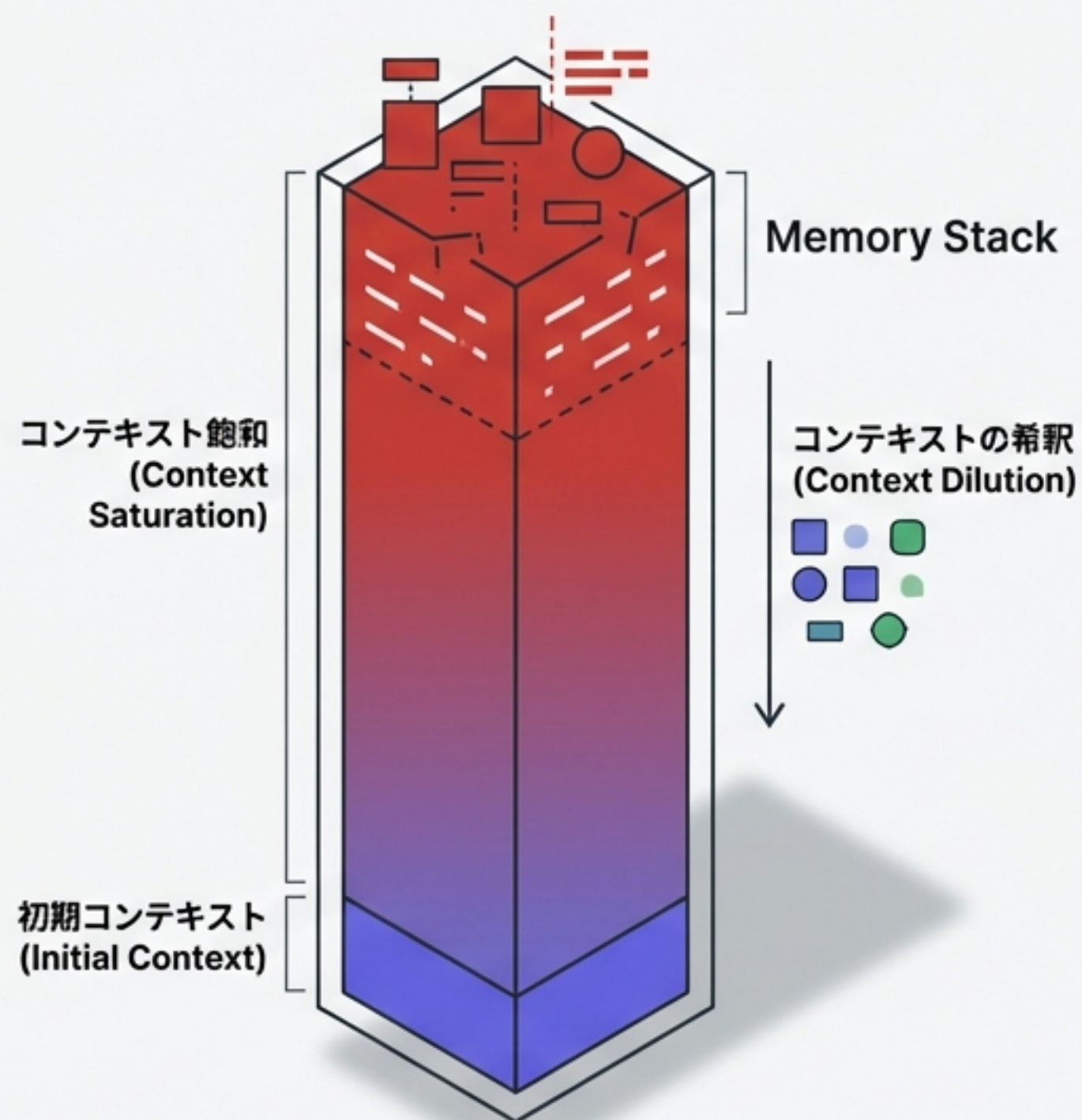
AIを「使う」から「率いる」へ。自律型開発チームの構築とオーケストレーション

単一のエージェント運用から、役割分担（Separation of Concerns）された専門チームによる契約駆動開発へ。



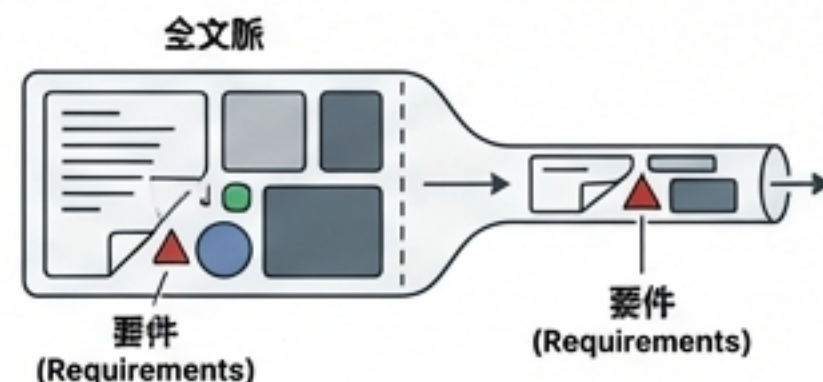
単一エージェント（All-in-One）の限界とリスク

大規模なマイクロサービスやモノレポ環境において、一貫処理型のアプローチは限界を迎えています。



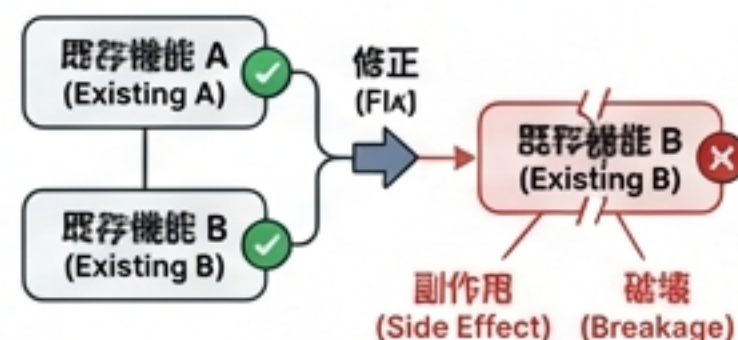
⚠ コンテキストの希釈 (Context Dilution)

全文脈を1つのセッションで処理するため、重要な要件が埋没する。



⚠ デグレードのリスク (Regression)

修正による副作用（Side Effects）を自己検知できず、既存機能を破壊する。



⚠ 推論のループ (Hallucination Loops)

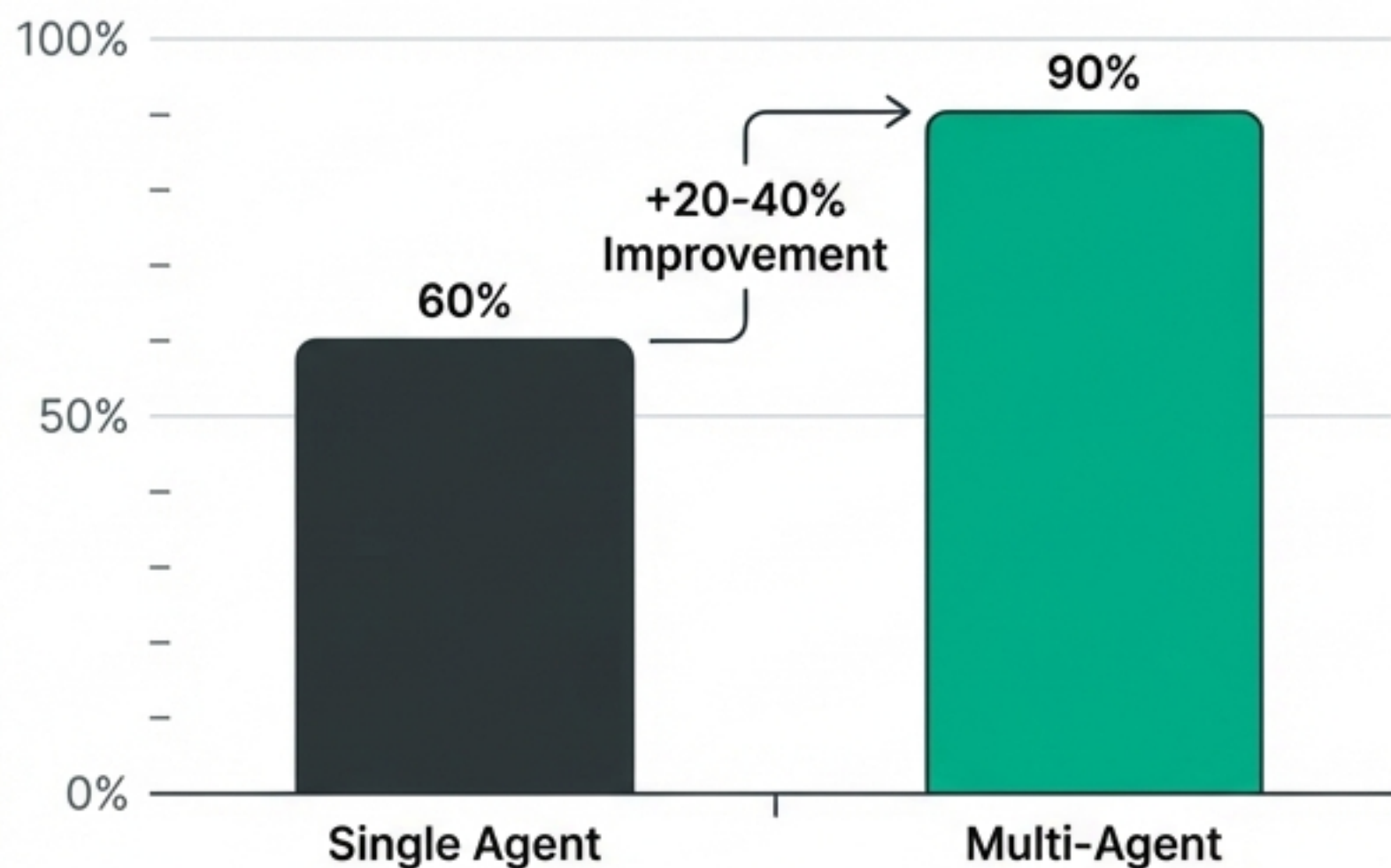
複雑性が増すと、修正→破壊→修正の無限ループに陥る。



戦略的転換：「関心事の分離」による品質向上

役割を物理的なエージェント単位で分割する「マルチエージェント・オーケストレーション」への移行。

Test Pass Rate



Execution Flow

Serial Execution



Serial Execution

Parallel Processing (7M Tokens Scale)

Security

Quality

Bugs

→ 7M Tokens Scale

✅ **安全性:** 「Read-Only」権限の強制により、意図しないコード破壊を構造的に防止。

チーム構成：あなたの新しい専門スタッフ



Explorer (調査役)

Mission: コードベースの探索、依存関係の特定、計画策定。

```
sandbox_mode = "read-only"
```



Worker (実装役)

Mission: 計画書に基づく実装、テスト作成。

```
sandbox_mode = "workspace-write"
```

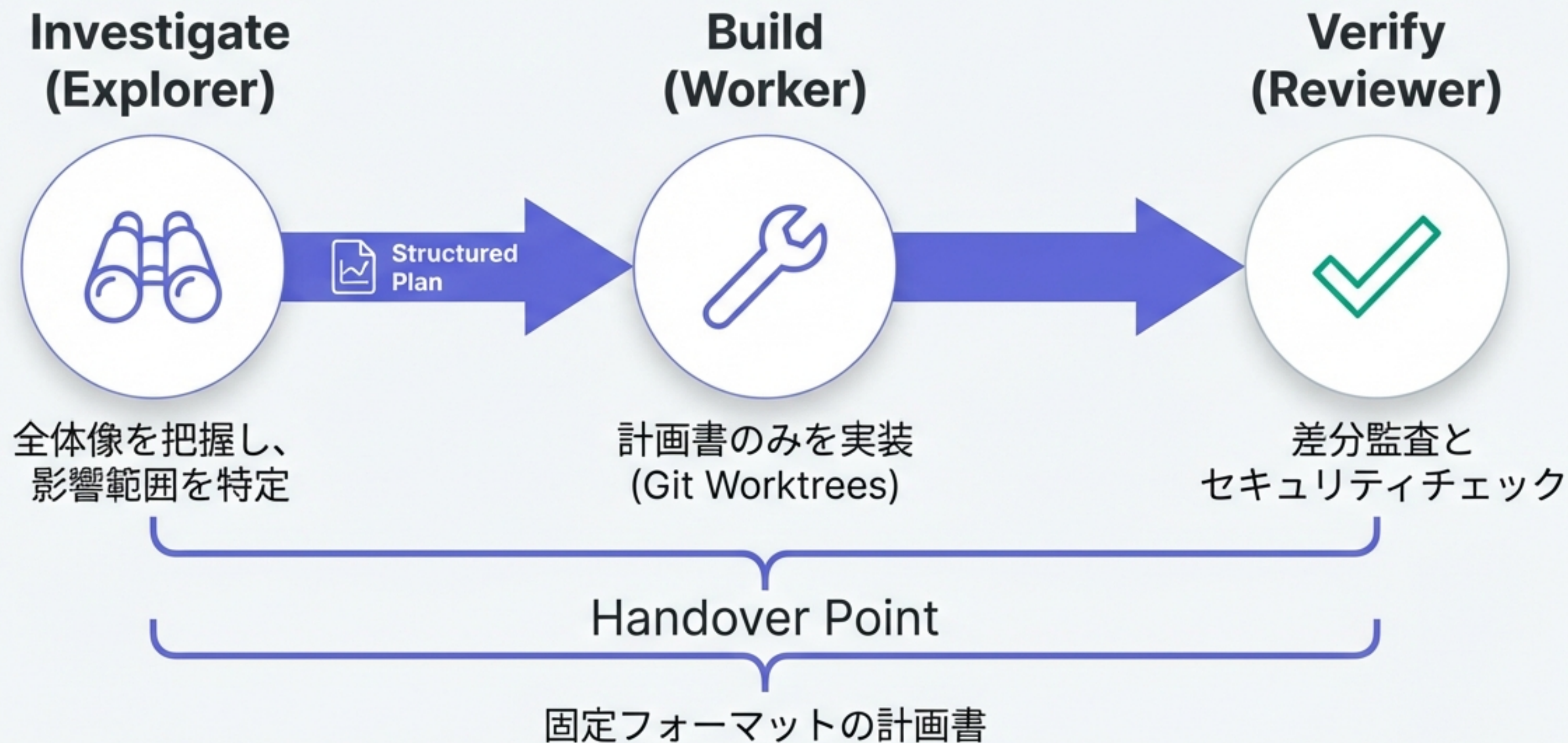


Reviewer (監査役)

Mission: セキュリティ、互換性、品質の独立監査。

```
read-only
```

黄金サイクル：調査 → 計画 → 実装 → 検証



Explorer（調査役）：5つの必須出力要件

調査結果の精度が、実装の品質を決定します。以下の5項目を含む成果物の出力を義務付けます。



Worker（実装）と Reviewer（監査）：職務分掌の徹底

Worker Strategy



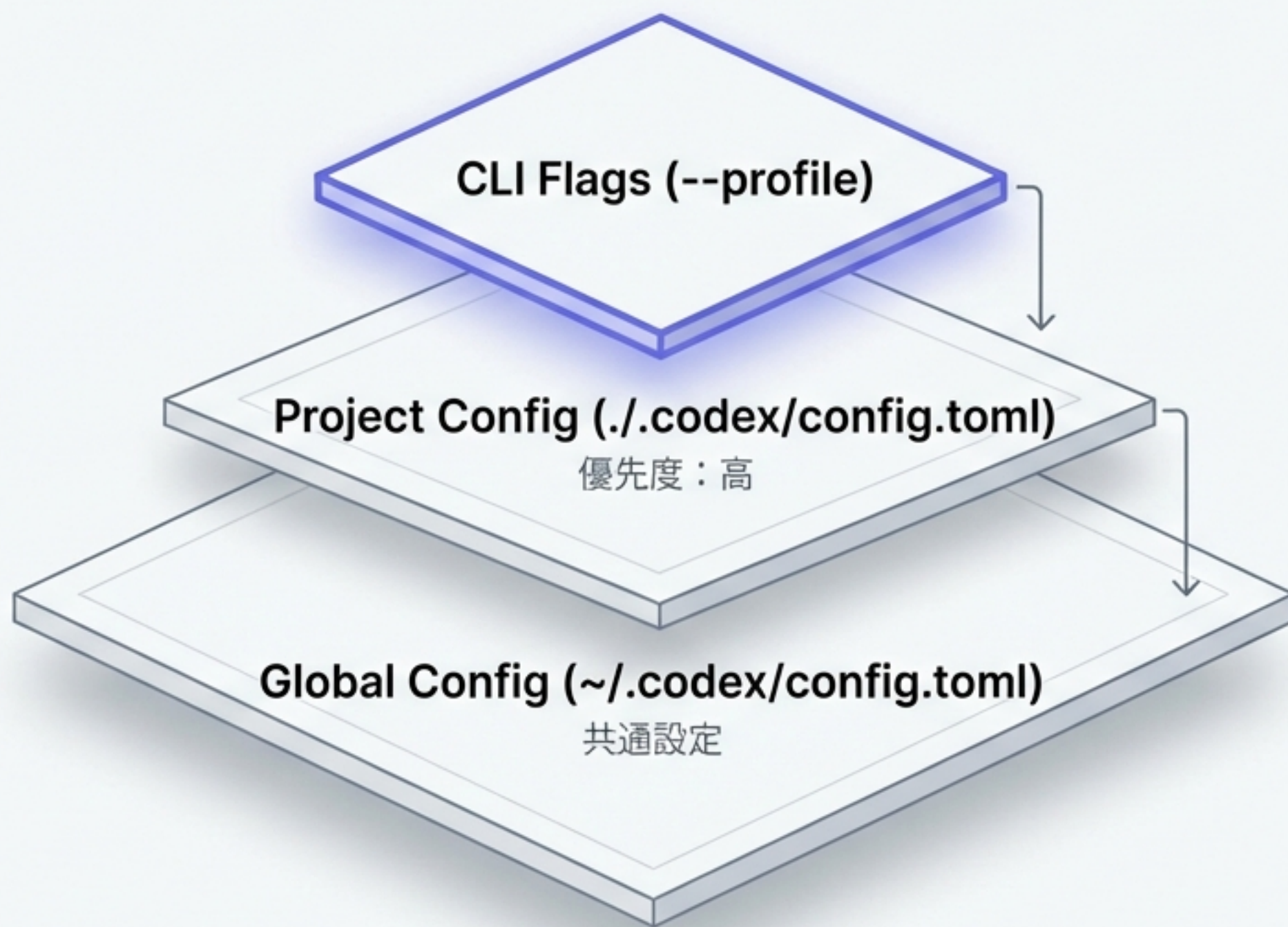
- Input: Explorerの計画書のみ
- Constraint: **最小差分** (Minimal Diffs)

Reviewer Strategy



- **SoD (Segregation of Duties)**: 実装者がレビューを行わない
- Focus: **脆弱性**、**テスト不足**、**後方互換性**

システム基盤：config.toml による階層的設定



```
[features]
multi_agent = true
max_threads = 12
```

推奨スレッド数: 4~12 (APIレート制限に注意)

実装詳細：カスタムロールの定義

config.toml (Parent)

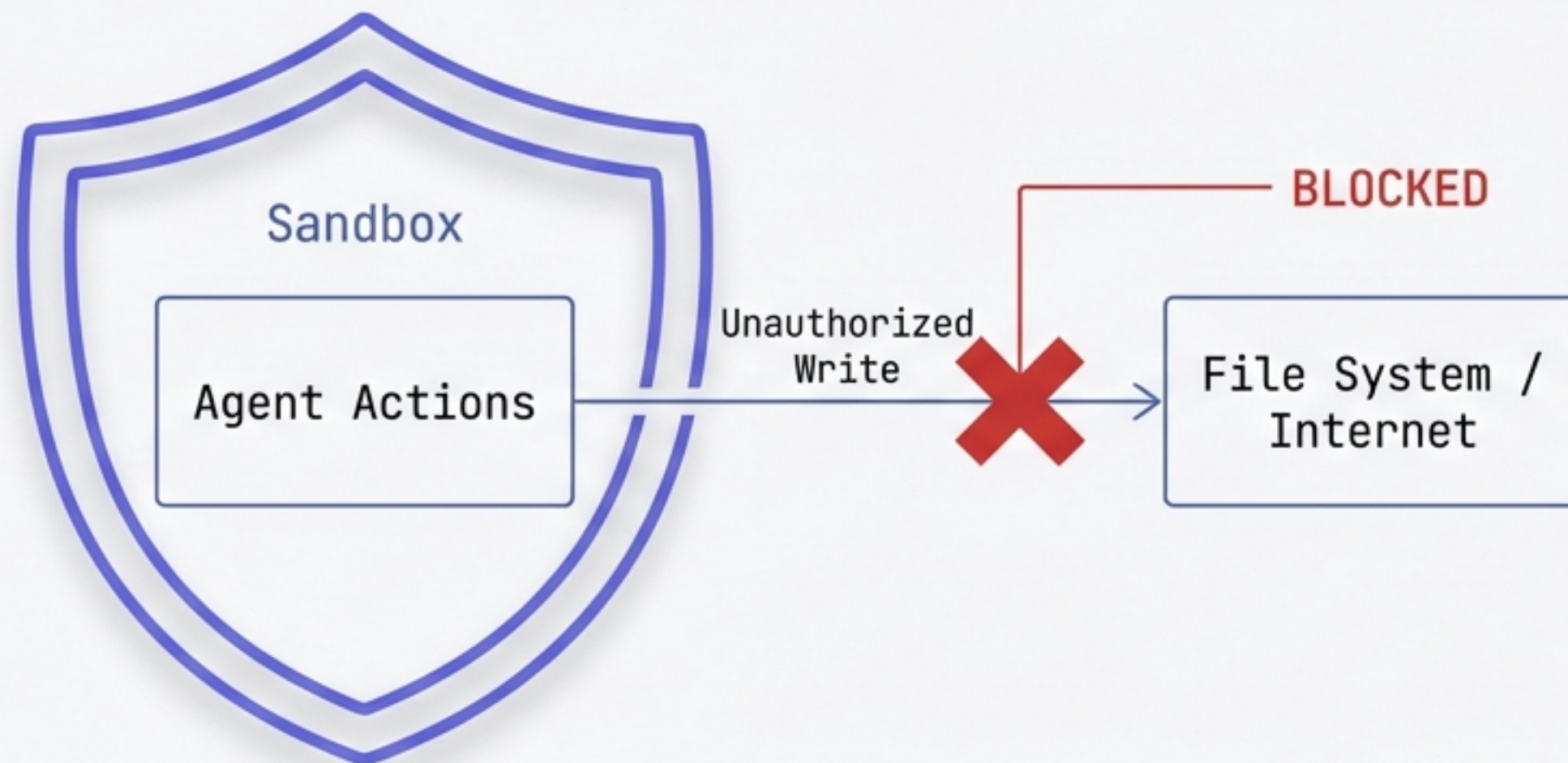
```
1 [agents.explorer]
2 description = "コードベース調査専用"
3 config_file = "agents/explorer.toml"
```

agents/explorer.toml (Child)

```
1 model = "gpt-5.3-codex-spark"
2 sandbox_mode = "read-only"
3
4 developer_instructions = """
5 You are the investigator...
6 Output sections exactly:
7 1) Services...
8 """
```

`config\_file` で設定を外部化し、`developer\_instructions` で役割を厳密に定義する。

ガバナンス：サンドボックスと非対話型承認

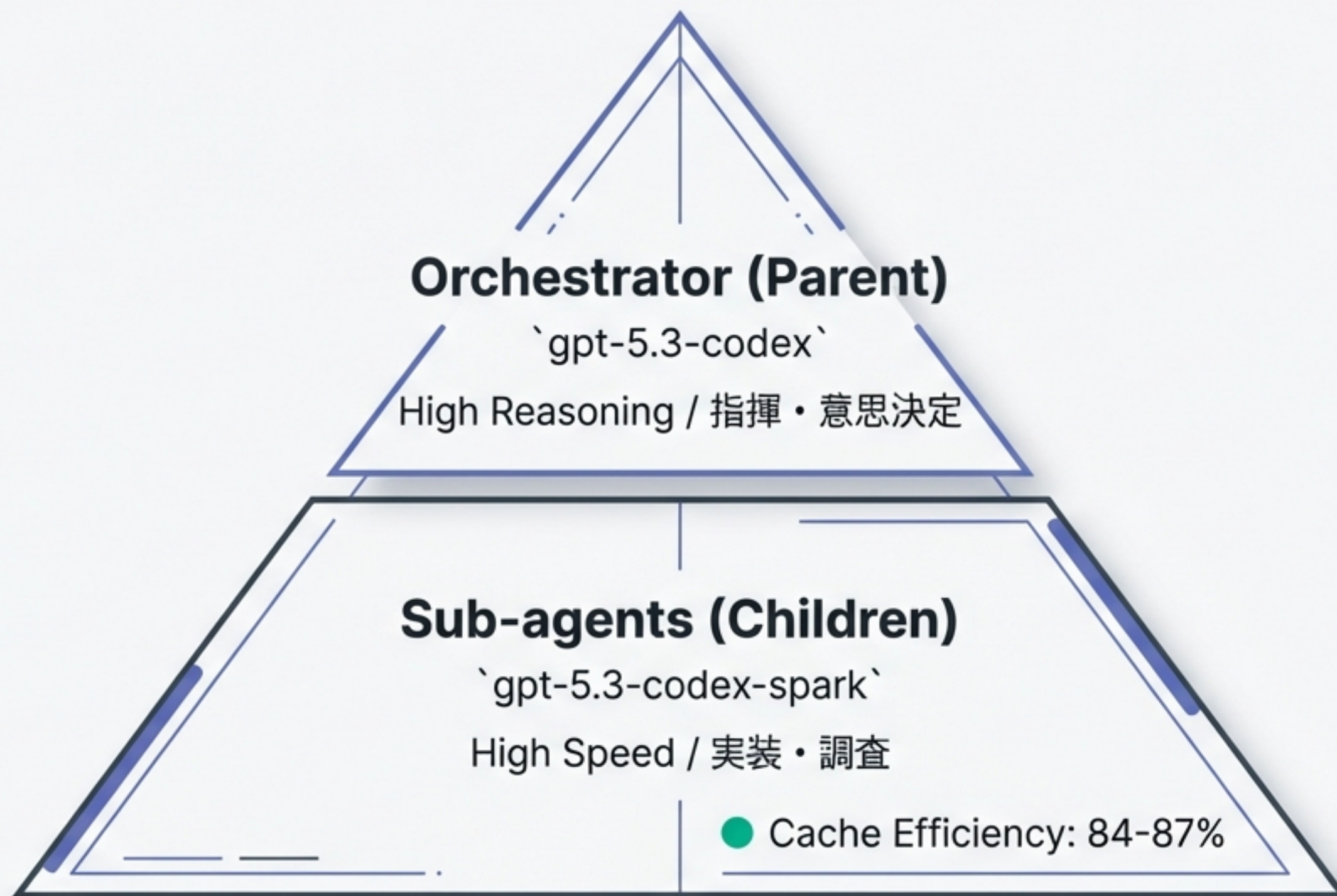


Read-Only:
Explorer, Reviewer
(ファイル変更不可)

Workspace-Write:
Worker
(許可されたディレクトリのみ)

Fail-Safe: 非対話型承認
(Non-interactive Approval)
- 権限外の操作は即座に失敗し、
親に報告される。

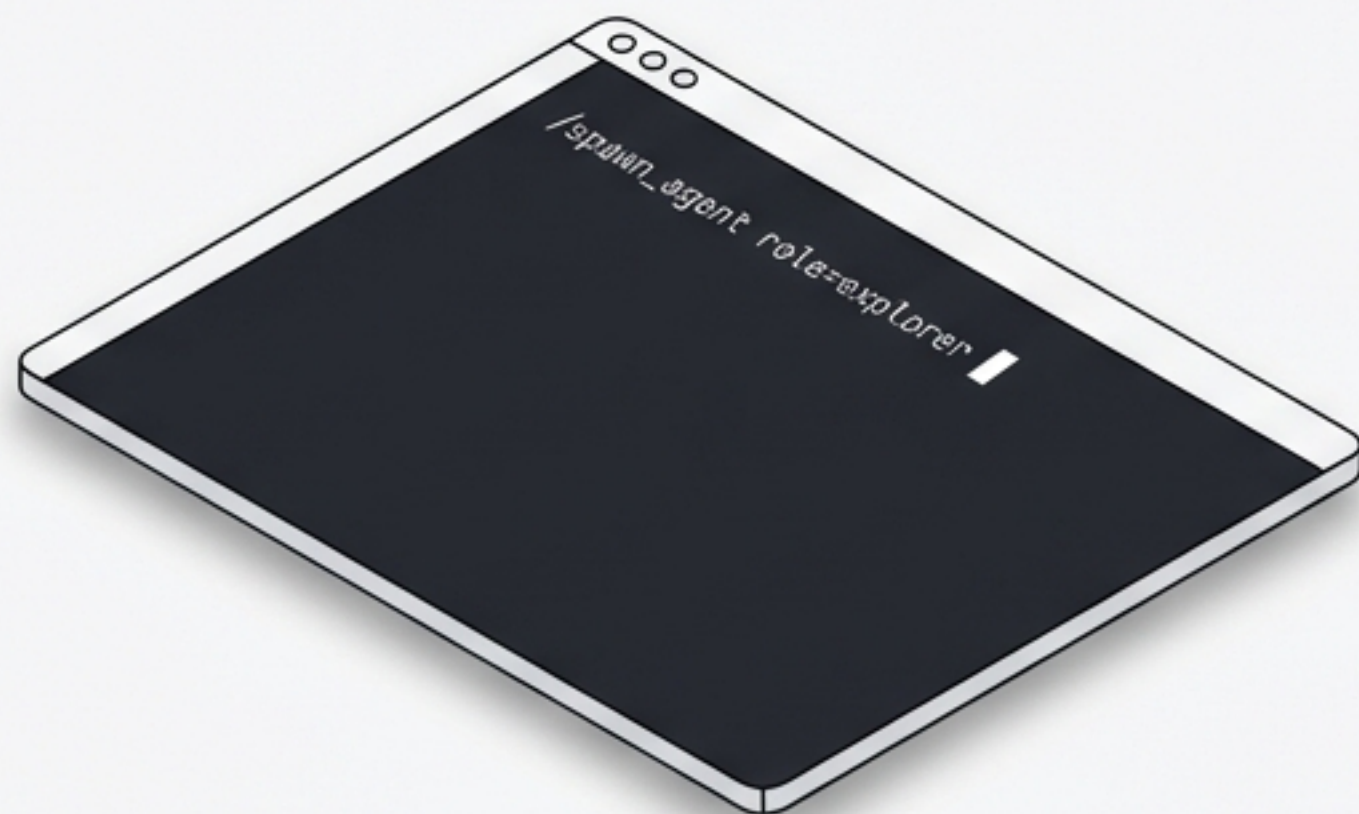
インテリジェンス・ティアリング：コストと速度の最適化



運用インターフェース：CLI と macOS App

Codex CLI

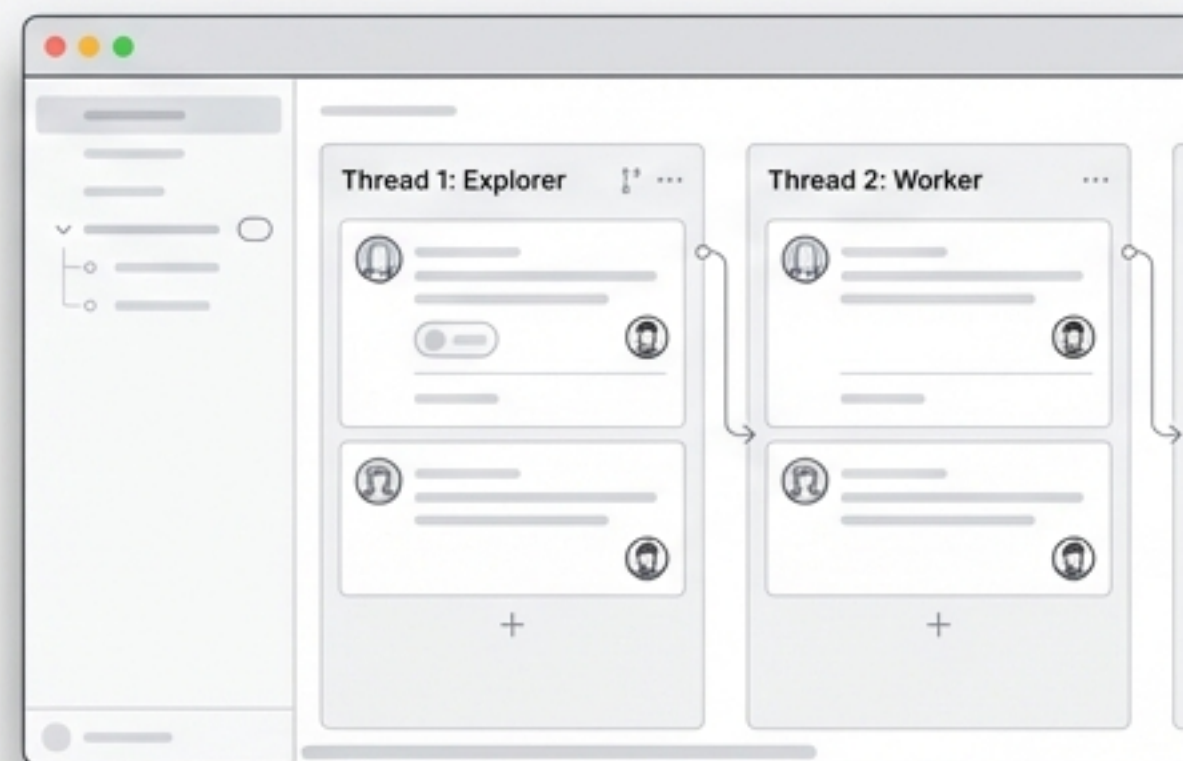
JetBrains Mono



自動化、厳密なワークフロー制御。

macOS App

JetBrains Mono



視覚的な監督、並列スレッド管理
(Git Worktrees support)。

トラブルシューティングと監査

Issue: Stuck / 処理停止

```
/agent list  
/agent close
```

リセットしてリソース解放

Issue: Rate Limits (429)

```
max_threads = 4
```

スレッド数を調整

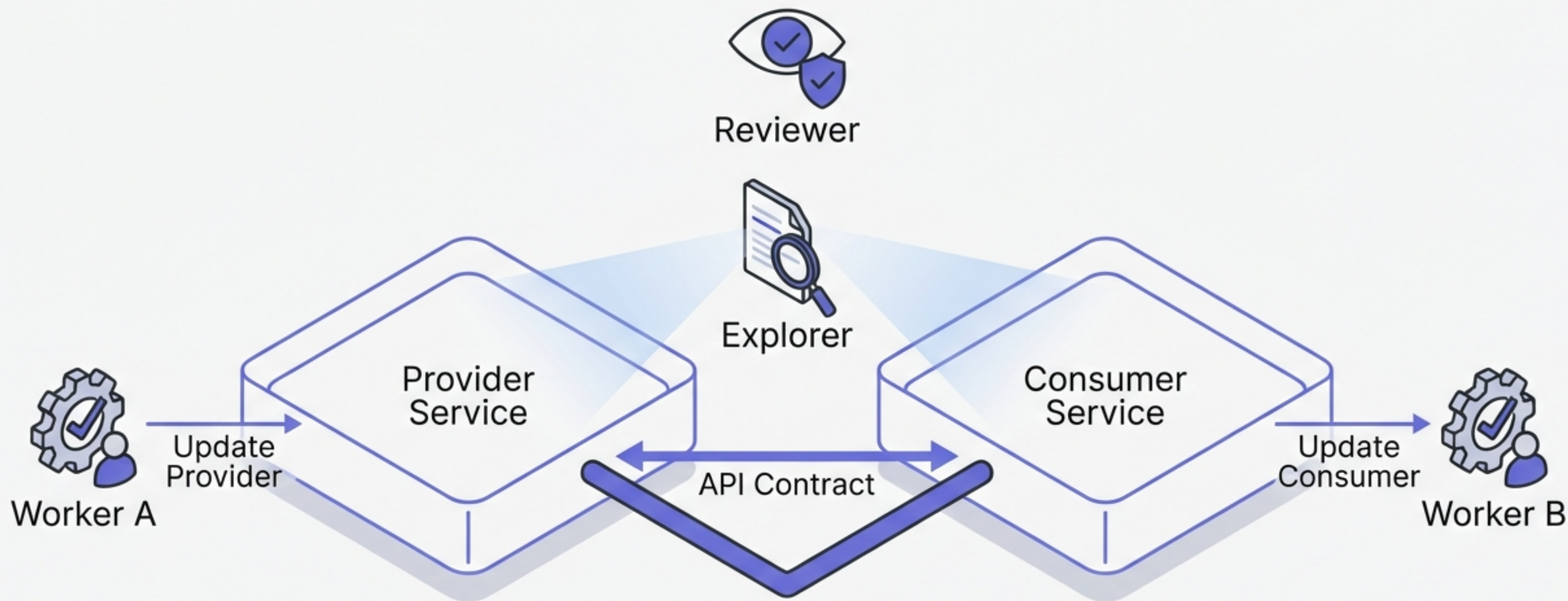
Audit Logs

```
~/.codex/sessions/*.jsonl
```

thread.started, file_change イベントを追跡

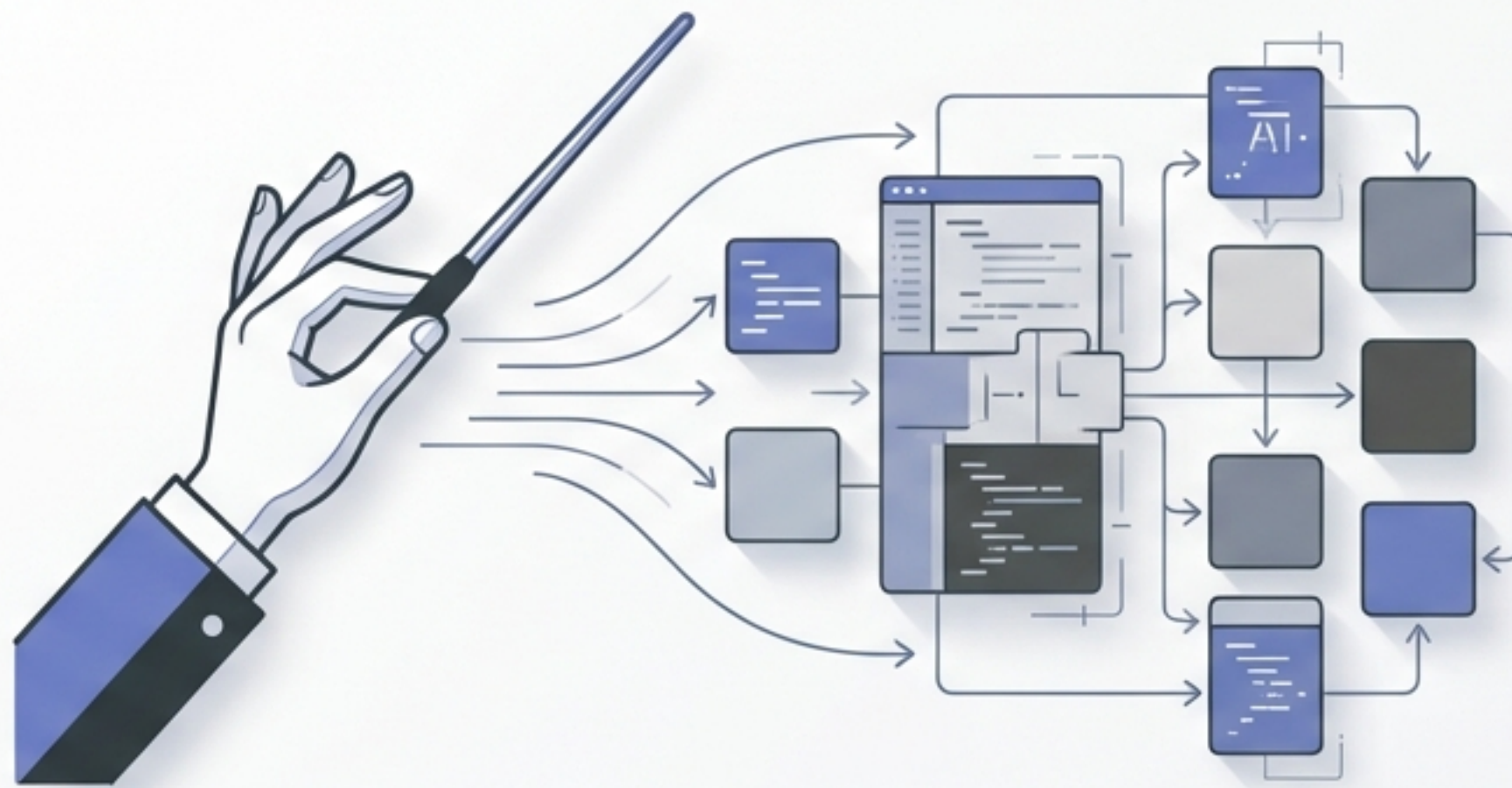


実践ユースケース：マイクロサービスの契約変更



「変更境界＝サービス境界」。サービスの境界を越える変更を安全に管理。

結論：実装者から「アーキテクト」へ



From: コードの細部に埋没する「作業者」

To: 専門性と権限を設計し、チームを監督する「設計者」

Architect your agents. Define the boundaries. Lead the team.

`config.toml`を定義し、あなただけのAI開発チームを編成してください。